

Where a Decision Model Fits in an Agent Harness

Amplifier Michael J. Jabbour David Koleczek

Microsoft, Office of the CTO

October 7, 2026

Abstract. A *system-1 decision model* answers an AI agent’s small typed questions (which model should run this session, may this file be read, is this action safe) in a fraction of a second. We measured where such models fit in an agent harness, using judge benchmarks, paired multi-turn sessions (one study preregistered on 60 fresh scenarios) and 19 days of production telemetry. *First*, Jev, a hosted typed-decision service, was the best price–quality choice when cost and latency matter (55/63 correct at \$18.41 per million decisions): dearer models were 8–11 points more accurate, which the preregistered corrected tests could not confirm, at many times the price; post hoc, OpenAI’s Decisions API scored the same or higher on every split at 1.3–1.7× the price but automated fewer real reads. *Second*, routing a session to a cheaper model paid only on the much dearer host once caching was counted: 0.581× the plain host on Fable 5.1 (about 0.75× with the review safeguard we recommend, replayed) and 1.255× on Opus 5.5. *Third*, configure once per session: deciding once cost 9–13 % less than re-deciding every turn, and a one-line rule matched the decision model at session start. *Fourth*, a deterministic guard removed all side-effect errors in risky actions; editable criteria remain untested. *Fifth*, telemetry measures latency and usage, not accuracy. Session results use one provider’s models, scripted tasks and list prices.

1 Introduction

An AI coding agent spends nearly all of its money on calls to a large *host model*. Between those calls sit many small choices: which model runs this session, may a file be read without asking the host, how much should the model reason, is an action safe. A decision model answers such typed questions with a probability per allowed answer in tens to hundreds of milliseconds. We asked where it earns its place in a harness, and where something simpler does as well. Routing single queries to cheaper models is well studied [1, 2]; an agent session is many calls over one cached, growing context, so we measured whole sessions in pairs and tested the decision model against a deterministic rule. The report is organised around five takeaways:

1. Jev is the best price–quality decision model we tested; OpenAI’s Decisions API is the challenger (Section 3; preregistered and post hoc).
2. Route to a cheaper model only when the host is much more expensive once caching and request volume are counted (Section 4; preregistered, two hosts).
3. Configure model and effort once per session; change only after the cache has expired (Section 5; once preregistered, expiry exploratory and for effort only).
4. Guard risky actions with a deterministic rule plus the model’s plain-text policy; editability is untested (Section 6).
5. Real telemetry measures latency and usage, not accuracy, and showed routing applies far less often than in the lab (Section 7; exploratory, one owner).

Section 2 describes setting, data and measures once; Section 8 gives the configuration and Section 9 ranks the takeaways. Every supporting result is in the supplement.

2 Setting, data and measures

The harness and the decision interface. The harness is an agent runtime with a pluggable orchestrator. At chosen points (session start, before a file read, before a computer-use action) it asks a decision model a typed question: a short state, a question and two to four allowed answers. The model returns a probability per answer. The harness acts automatically only when the top probability is at least 0.90, leads the runner-up by at least 0.20 and arrives within 3 s; otherwise it falls back to the host model. The session-start decision chooses the session’s model and reasoning effort once; the shipped default can also use a one-line rule instead of a model (route to the cheaper model unless the workspace has more than 300 files) and a price gate that keeps the host whenever routing cannot pay at current prices.

Models and prices. The hosts were Fable 5.1 and Opus 5.5; the routing target was Sonnet 5. At list prices (US dollars per million tokens: input, output, cache read, cache write) Fable costs 10.00, 50.00, 0.25 and 12.50; Opus 4.00, 20.00, 0.20 and 5.00; Sonnet 3.00, 15.00, 0.30 and 3.75. Sonnet’s cache reads cost more than either host’s. The decision models were Jev (a hosted typed-decision service), GPT-6 Luna and GPT-6.1 Sol prompted for probabilities, Cloudflare’s Clef and Clef-Flash, OpenAI’s Decisions API (Luna behind a native typed-decision endpoint) and seven local models.

Data. Table 1 lists the sources and what each can and cannot show. The *production telemetry* is one owner’s harness: 77 sessions carried an explicit production tag and 321 were counted as production because they carried no test tag; *test* traffic is sessions tagged as tests or evaluations, including our own benchmark runs. The *constructed* judge cases were labelled by language models under a written guide and checked by two further language-model reviewers, so labels follow the guide rather than any operator’s preference. The *real-decision* cases were rebuilt from earlier agent sessions by fingerprint matching, with the state clipped to the decision model’s 2,048-character input budget. The *holdout study* (S1) panel was disjoint from the main campaign and was smoke-tested before preregistration: one plain-Sonnet session per scenario found prompt or grader defects in 30 of the 60 scenarios, which were repaired (genuine agent mistakes were kept) before the confirmatory run. Every pair of sessions starts from the same frozen workspace at the same time, each with its own cache key, so the two arms differ only in the configuration under test.

What makes this evidence strong. Sessions are paired and start together from one frozen workspace; the central routing result was preregistered and replicated on 60 fresh scenarios (1,224 sessions); every number is regenerated from committed evidence; and results that cut against our own default (the Decisions API, a corrected projection) are reported in the body. The common limits are also stated here once: session results use one provider’s models (two hosts, one routing target), scripted tasks of at most 16 turns, automated quality checks rather than human judgement, and list prices rather than invoices.

Measures. *Cost* is the list-price cost of a session’s measured tokens, with the shared tool-definition prefix repriced from the cache-write to the cache-read rate (a 0.45 % adjustment) plus the decision model’s estimated charge. A *cost ratio* compares paired sessions of the same task; we report the *geometric mean* of per-scenario ratios with 95 % scenario-cluster bootstrap intervals (90 % for equivalence tests). *Quality* is the *turn-pass* fraction: the share of a session’s scripted turns whose automated checks passed; non-inferiority means its 95 % lower bound stays above -0.05 . For decision models, *accuracy* is the share of cases answered correctly after the fallback rule, *wrong automatic* the share

Table 1: Data sources, what each shows, and its main weakness. “Preregistered” means hypotheses, thresholds and analysis were committed before the data existed.

Source	What it is	Strength / weakness
Production telemetry	464,524 events over 19 days; 398 production sessions of one owner	Real usage and latency / one user, production partly a residual class, no correctness labels
Constructed judge benchmark	90 development and 63 preregistered holdout cases (read, search, select, computer use), 3 repetitions	Known answers, paired across judges / labels written by language models
Real-decision benchmark	42 held-out read decisions rebuilt from earlier agent sessions	Realistic states / small; rebuilt, not captured
Main paired campaign	1,036 sessions on 70 scripted scenarios, 8–16 turns, both hosts	Whole-session cost and quality / estimator written after the data
Holdout study (S1)	1,224 sessions on 60 fresh scenarios	Preregistered, fresh tasks / scripted tasks, list prices
Effort follow-ups	Medium vs default effort on Sonnet and Fable	Clean effort contrast / 23 scenarios each
Decisions API run	The native endpoint against an in-run Jev reference on all four judge splits	Paired in time / post hoc, slow network

acted on wrongly without fallback, *coverage* the share acted on automatically, and *latency* the client wall time. Labels mark each result [preregistered], [post hoc] or [exploratory].

3 Choosing the decision model: Jev is the best price–quality choice

Claim. Among the system-1 models we tested, Jev is the best price–quality tradeoff when cost and latency matter: it is the cheapest and fastest accurate judge, and no dearer model was shown more accurate in the preregistered test. OpenAI’s Decisions API is the main challenger (post hoc: the same or higher accuracy at 1.3–1.7× the price, with fewer automated real reads).

Evidence. On the preregistered constructed holdout, Jev answered 55 of 63 cases correctly at \$18.41 per million decisions with a client p95 of 216–222 ms (Figure 1). GPT-6 Luna answered 60 at 2.2× the cost and a p95 of 1.8–1.9 s; GPT-6.1 Sol answered 62 at 48× the cost and 3.6–4.7 s. That is an apparent accuracy premium of 8–11 points (+7.9 and +11.1; 5 and 7 cases, never wrong where Jev was right), with paired bootstrap intervals of +1.6 to +15.9 and +3.2 to +19.0 points, that the corrected preregistered tests could not confirm (Holm $p = 0.16$ and 0.09 , over a 13-arm family) [preregistered]. Clef scored 57 against an in-run Jev’s 56 at 3.2–4.4× the cost [post hoc]; Clef-Flash scored 50. The seven local models were less accurate and none met the preregistered safety bar. No challenger met the preregistered rule for replacing the incumbent (better accuracy or fewer misfires, p95 within 500 ms, at most twice the cost).

The Decisions API, post hoc. OpenAI’s native typed-decision endpoint was run after the preregistered work, paired in time with an in-run Jev reference. It scored 88 against 86 development cases, 59 against 55 holdout cases, 16 against 16 and 30 against 23 real read decisions; only the last difference survived Holm correction ($p = 0.039$, exploratory). On real read decisions it automated fewer (7 against 15 of 42), so part of its accuracy gain is more fallbacks to the host. Its misfire rate was never higher than Jev’s and it cost 1.3–1.7× as much per decision. Its servers answered in 136–179 ms at p95, but on a network that was slow for both judges the client saw seconds; 9 of its 1,296 requests and 4 of Jev’s failed in transport.

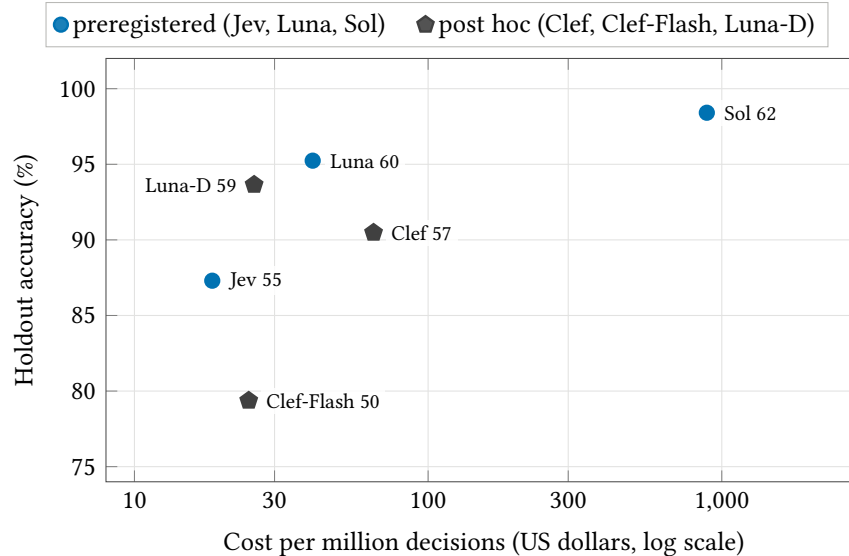


Figure 1: Jev is the low-cost point on the accuracy–cost frontier. Holdout accuracy against cost per million decisions for the cloud judges with a price; labels give correct answers out of 63. Local models are omitted (no priced compute).

How to read it. Up is more accurate, left is cheaper. Luna and Sol are more accurate at about two and fifty times Jev’s price; Clef-Flash is cheaper per point than neither. Luna-D, the post hoc Decisions API run, sits above Jev at 1.3–1.7× the price.

On real read decisions. On 42 held-out read decisions, 0 of 10 judges met the preregistered usefulness rule; Jev took the most correct automatic reads (10 of 13) but also made 5 wrong ones. No model was safe enough to act alone there.

For practitioners. Use Jev as the default: it is the cheapest accurate option and fast enough for interactive use on a clean network path. Pay for a dearer judge only where 8–11 points of possible accuracy are worth many times the price, and re-test the Decisions API, which matched or beat Jev on our cases at 1.3–1.7× the price. Keep a fallback to the host on real read decisions.

Details: Supplement Section [A2](#).

4 Route only when the host is much more expensive, caching included

Claim. Routing a session to a cheaper model saves money only when the host is much more expensive once you count what a session actually buys (mostly cache reads and writes) and the extra requests the cheaper model makes. On Fable, routing alone cost 0.590× the plain host and the full preregistered package 0.581×; on Opus it cost 1.255–1.457×. Both directions replicated on fresh scenarios.

Why we measured whole sessions. Per-token list prices misled. Opus’s input price is above Sonnet’s, yet an agent session is mostly cached context, and Sonnet’s cache reads cost more than Opus’s. The only honest test was to run the same task twice, once on the plain host and once routed, and compare the bills.

Evidence. [Table 2](#) prices one average request of the plain-host sessions (88,516 cache-read, 5,376 cache-write and 541 output tokens, measured over 280 sessions) at each model’s list prices. Sonnet costs 0.47× Fable per request but 0.99× Opus. Sonnet sessions also made more requests (1.11× on Fable’s tasks, 1.38× on Opus’s), so the predicted routing ratios are 0.524 and 1.365. The measured

Table 2: Sonnet is half Fable’s price per request but Opus’s equal. Cost of one average plain-host request at list prices, its composition, and the routing ratio predicted from the per-request price and the measured request multiplier [exploratory].

Model	\$ per request	share of request cost (%)			routing to Sonnet		
		write	read	output	per request	requests	predicted
Fable 5.1 (host)	0.1164	58	19	23	0.47	1.11	0.524
Opus 5.5 (host)	0.0554	49	32	20	0.99	1.38	1.365
Sonnet 5 (routing target)	0.0548	37	48	15	–	–	–

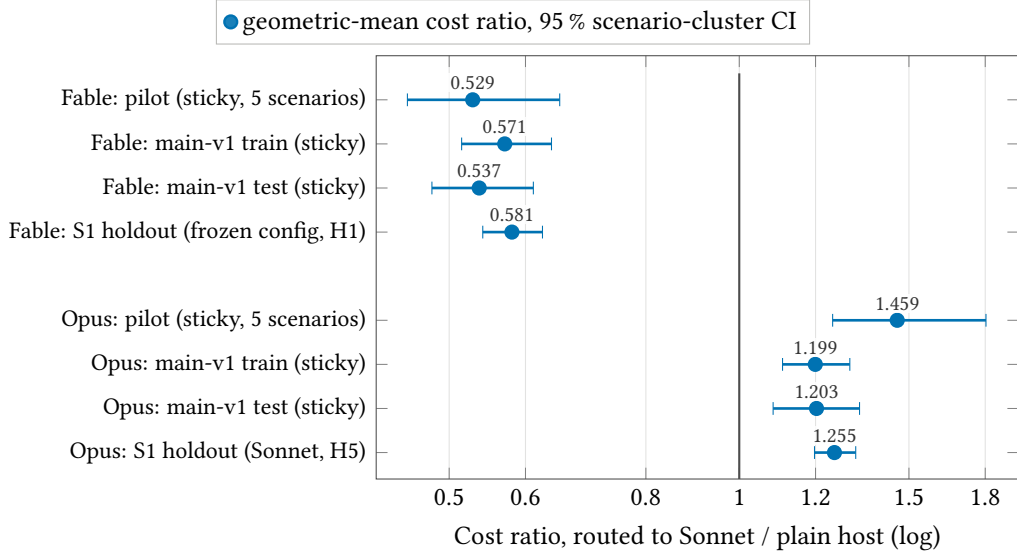


Figure 2: The same direction in every study. Routing to Sonnet against the plain host, per host: the pilot and the main campaign’s training scenarios [exploratory], its test scenarios (sticky arm, all pairs) and the preregistered holdout study (the frozen Fable configuration; Sonnet on Opus).

How to read it. Each dot is a geometric-mean cost ratio with its 95 % interval on a log scale; left of 1 saves money. Every Fable row sits near one half and every Opus row above 1; the small Opus pilot is the highest.

ratios agree in sign and size (Figure 2). In the main campaign plain Sonnet against plain Fable, both at default effort, cost $0.590\times$ ($0.529\text{--}0.663$), the cleanest model-only contrast; against plain Opus it cost $1.457\times$ ($1.280\text{--}1.666$) [preregistered] (all pairs; the quality-filtered pair reading is in the supplement). On 60 fresh scenarios the preregistered Fable configuration (routing plus medium effort) cost $0.581\times$ plain Fable ($0.542\text{--}0.625$) and routing on Opus cost $1.255\times$ ($1.197\text{--}1.321$) [preregistered]. Holding token counts fixed, routing on Opus would only break even if Opus’s cache reads cost about \$0.38 per million tokens instead of \$0.20 [exploratory].

Because two hosts and one cheap model were measured, “much more expensive” is a modelled threshold: no host lay between Opus’s one-to-one and Fable’s two-to-one per-request price against Sonnet.

For practitioners. Before routing, price an average request of your own sessions on both models, including cache reads and writes, and multiply by how many more requests the cheaper model makes. Route only when the result is clearly below one; a price gate that does exactly this predicted both of our directions.

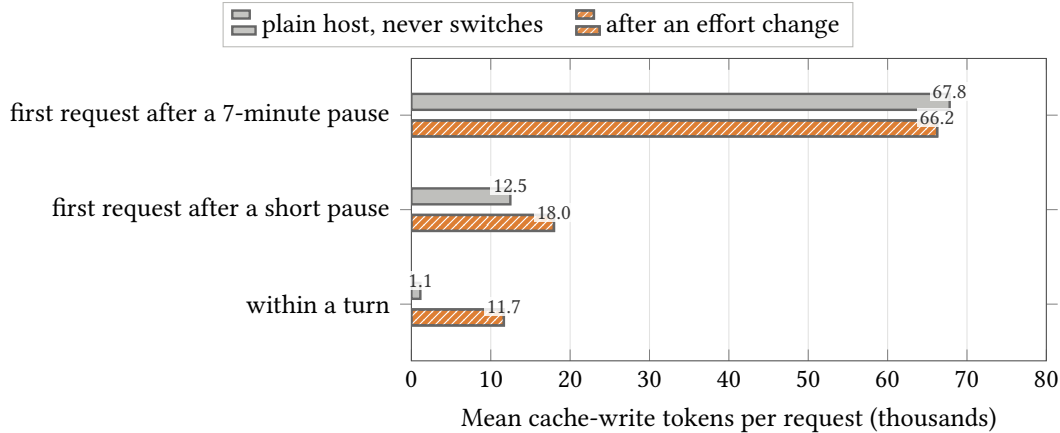


Figure 3: A switch rewrites the cache unless the cache has already expired. Mean cache-write tokens per request on the plain host, which never switches, and on requests made after an effort change, by position in the session (main campaign re-analysis).

How to read it. Compare the two bars in each row. Within a turn the switch multiplies the write cost; after a seven-minute pause the two bars are about the same length ($n = 24$ each), consistent with the expired cache being rewritten whatever the configuration.

Details: Supplement Sections [A3](#) and [A4](#).

5 Configure once per session; switch only when the cache has expired

Claim. Choose the model and reasoning effort once, at the start of the session, from the task and its initial context, and do not switch while the prompt cache is live. If a switch is ever worth it, make it after the cache has expired (measured for effort changes only, with five-minute caches). The start-of-session choice does not need a decision model: a one-line rule made the same choice.

Evidence. Every switch of model or effort invalidates the provider’s cached prefix, so the next request pays to write it again ([Figure 3](#)). Within a turn, a request after an effort change wrote 11,650 cache tokens on average against 1,133 for the plain host, which never switches ($10.3\times$; $n = 486$ and $1,331$) [[exploratory](#)]. In the main campaign, deciding the model once per session cost $0.87\times$ ($0.78\text{--}0.98$) of re-deciding it every turn on Fable and $0.91\times$ ($0.85\text{--}0.97$) on Opus (all 46 paired triplets per host; the arm that decided once still applied a phase-based effort map in sessions that kept the host). The preregistered criterion was only that deciding once cost no more than $1.05\times$ of re-deciding; the size of the saving is descriptive [[preregistered criterion](#)]. After a pause longer than the five-minute cache lifetime the picture changes: the first request rewrote the history whatever the configuration ($5.8\times$ the writes after a short pause on Fable, $6.2\times$ on Opus), and an effort change at that point added no detectable write cost (66,235 against 67,820 tokens; $n = 24$ each, small sample, no interval computed) [[exploratory](#)].

The start-of-session choice needs no model. In the preregistered holdout study, letting Jev choose the session’s model and letting the rule “route unless the workspace has more than 300 files” choose it were equivalent within the preregistered $\pm 5\%$ margin: $1.008\times$ (90 % CI $1.000\text{--}1.019$), and the two disagreed on only 4 of 120 scenario-reps [[preregistered](#)]. Live decide-once sessions cost what the reconstruction from pinned sessions predicted ($1.014\times$, 90 % CI $0.996\text{--}1.032$) [[preregistered](#)].

How much cheaper a new configuration must be to justify a mid-session switch was not measured, and a model switch timed to an expired cache was not isolated.

For practitioners. Decide once per session with a rule and a price gate. Keep model and effort fixed while the cache is warm. If a session idles past the cache lifetime, that is the cheap moment to change configuration, and only for a clearly cheaper one.

Details: Supplement Sections [A3](#) and [A4](#).

6 Risky actions: a deterministic guard works; editable criteria are untested

Claim. For actions with side effects (buying, sending, deleting, deploying), use the decision model’s natural-language criteria together with a deterministic guard. What we measured supports the guard; the model’s advantage over a fixed classifier, that its criteria are plain text a user can edit, is a promise we have not yet tested.

Evidence. In the constructed benchmark’s computer-use cases, every judge, Jev included, clicked “buy” in a purchase case when the task asked for it and no policy said otherwise: models follow the task unless the risk rule is stated. A deterministic guard that matches a short list of irreversible verbs on the proposed action removed every side-effect wrong automatic action for all 9 applicable judges on the holdout (38 to 0), at a coverage cost of 1.6–7.9 points and no accuracy loss [[preregistered](#)]. Jev itself made no such error on the holdout computer-use cases at the shipped gate; the guard matters for the other judges and for Jev on the development cases (5 side-effect errors there). A one-sentence natural-language risk clause in the prompt was not confirmed overall (22 to 20 targeted errors; confirmed for 1 of 3 applicable judges) [[preregistered](#)]; on the development cases it removed all of Jev’s side-effect errors [[exploratory](#)]. The shipped computer-use tool hard-codes both the clause and the guard, so its criteria are not yet user-editable; editability, live risky actions and a comparison with a trained risk classifier are untested.

For practitioners. Put the guarantee in a deterministic guard and the policy in plain text the model reads. Keep the confidence gate strict and let the host approve anything the model hands back. Treat editable criteria as the decision model’s potential advantage over a fixed classifier, and measure it before relying on it.

Details: Supplement Section [A2](#).

7 What real-usage telemetry tells you, and what it cannot

Claim. Production telemetry is the right instrument for latency, usage mix and how often routing actually applies, and the wrong one for accuracy or savings. In real use routing applied far less often than in the lab.

Evidence. One owner’s harness recorded 464,524 events over 19 days, 398 of them production sessions ([Table 1](#)) [[exploratory](#)]. A Jev decision took 159 ms at the median against 4.8 s for a host call, 30× faster (87× at p95; [Figure 4](#)). In production the workspace-size gate kept 49 % of sessions on the host, and Jev routed 34 % of the sessions it judged, against 89 % in the main campaign and 97 % of test sessions. Effort was switched 12 times in 12,094 production requests (against 828 in 3,752 test requests), and no production session ran on Fable, the host where routing pays. The bundle’s record of how often its proposals matched the host’s next action was a usage signal, not a correctness label: 0.2 % in production against 11.4 % in test traffic, because most host actions could not have matched by construction (the headline 7.1 % agreement was mostly such unmatchable decisions).

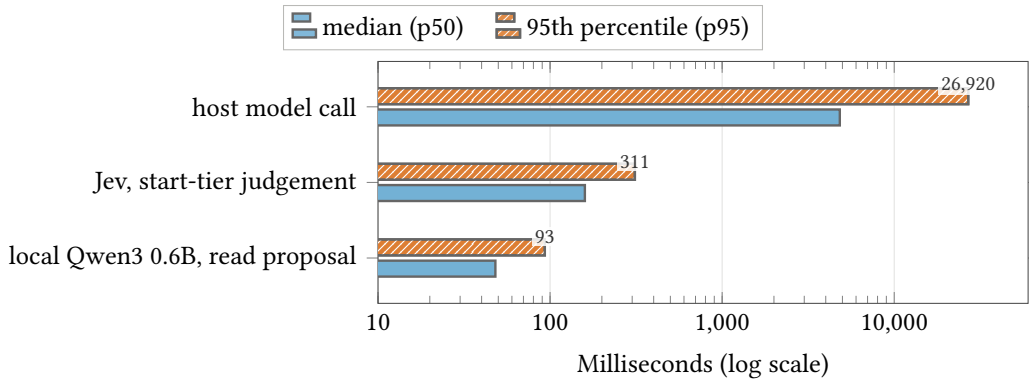


Figure 4: A decision call is about thirty times faster than a host call at the median. Latency of decision calls and of host model calls recorded in production and test traffic (log scale).

How to read it. Each row has a median and a 95th-percentile bar; values at the bar ends are p95 in milliseconds.

A corrected projection. Before S1 we projected that the recommended configuration would cost $0.83\times$ ($0.80\text{--}0.86$) of shipped defaults on this owner’s spend mix. That projection assumed medium effort saves about as much on Opus as on Sonnet, which S1 did not demonstrate ($0.992\times$). Recomputed on the same mix (53 % Opus, 46 % Sonnet spend), with Opus at default effort and Sonnet hosts at medium ($0.821\times$), the projection is $0.92\times$ ($0.90\text{--}0.94$) [exploratory]; the correction is documented in the supplement.

For practitioners. Instrument your harness for latency, the host-model mix and how often each gate fires. Do not read accuracy or savings from it: measure those with labelled cases and paired sessions. Expect real usage to route less often than a benchmark suggests.

Details: Supplement Section A5.

8 Recommendations

Table 3 summarises the configuration these results support. The same settings apply whether the decision is made inside the harness or by a small command-line tool before a third-party agent starts; the supplement gives ready-to-use configurations (Supplement Section A6).

Table 3: Recommended configuration per host model, at the list prices of Section 2.

Host	Session model	Reasoning effort	Evidence
Fable 5.1	Route to Sonnet, decided once by the workspace-size rule and the price gate; keep review and explain tasks on the host ^a	Sonnet at medium; Fable at medium when kept	Sections 4 and 5
Opus 5.5	Keep the host: the price gate never routes	Default (medium showed no saving)	Section 4
Sonnet 5	Already the cheap model	Medium ($0.821\times$)	Effort follow-up
Any harness	Decide once per session; keep the price gate on; use the rule, not a model, for the session start; use Jev where a decision model is needed and re-test the Decisions API; keep a deterministic guard on risky actions (Sections 3 and 6).		

^a In the holdout study, non-inferiority of Sonnet’s turn-pass was not established on review and explain tasks (lower bound -0.063 against -0.05), so a preregistered rule keeps them on the host. Without a decision model the shipped configuration identifies them with a keyword classifier; replayed on the holdout sessions this gives about $0.75\times$ of plain Fable, against $0.58\times$ without the safeguard (replayed, not run live).

9 Conclusion: what matters most

In order of how well the evidence supports them:

1. **Route only when the host is much dearer once caching is counted.** Routing to Sonnet cost $0.581\text{--}0.590\times$ on Fable and $1.255\text{--}1.457\times$ on Opus, in two independent scenario sets [preregistered]. A price check that weighs cache reads and writes and the cheaper model’s extra requests predicted both directions.
2. **Choose model and effort once per session.** Deciding once cost no more than re-deciding every turn [preregistered criterion], and switches rewrite the cache [exploratory]; a one-line rule made the start-of-session choice as well as the decision model [preregistered].
3. **Use Jev where a decision model is needed, and re-test the Decisions API.** Jev is the cheapest accurate judge; the dearer models’ accuracy edge was not confirmed [preregistered], and the Decisions API matched or beat Jev at higher price [post hoc].
4. **Guard risky actions deterministically.** The guard removed every side-effect error we measured [preregistered]; plain-text, editable criteria are promising but untested.
5. **Use telemetry for latency and usage mix, never for accuracy or savings** [exploratory].

A fast decision model therefore belongs where a cheap typed judgement changes an expensive or risky action and no rule does as well. The open question is decisions whose right answer changes during a session, such as escalating to the host or delegating a sub-task. *Scope:* these results come from one provider’s host models, scripted tasks of at most 16 turns, automated quality checks, list prices and one owner’s telemetry; the judge comparison spans several vendors but uses model-written labels.

Data and code availability. All analysis code, the evidence packages and the build that regenerates every number in this paper and its supplement are in the repository <https://github.com/michaeljabbour/amplifier-bundle-fast-decisions> at commit 3cd180b7523d; the evidence package IDs are listed in Supplement Table S1.

References

- [1] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous and I. Stoica. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665, 2024.
- [2] L. Chen, M. Zaharia and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176, 2023.